Day 41 — Line-by-Line Syntax Explanation

day41_convolutional_layers.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 6, Day 6 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (the convolution sum, the output-size formula, max pooling, or their backward-pass gradients), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the extended training work cycle, and the script's own real output graphs.

MATH box legend: × = matrix multiply / elementwise times W, b = filter weights/bias m = batch size k = kernel size C_out = number of filters (i,j) = an output position

Segment 0 — Imports and Global Setup (lines 31–37)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)
```

Identical setup pattern to Days 36-40: **np** for all array math, **matplotlib.use("Agg")** before importing **pyplot** so charts render straight to PNG files instead of a GUI window. No scikit-learn import this time — today's stripe dataset (Segment 6) is generated by hand rather than via a library function like **make_blobs**.

Segment 1 — Shared Helpers: ReLU, softmax, loss, He scale (lines 44–83)

MATH

$$d/dz[ReLU(z)] = 1 \text{ if } z > 0 \text{ else } 0$$

```
def relu(z):
    return np.maximum(0, z)

def drelu(z):
    return (z > 0).astype(z.dtype)
```

np.maximum(0, z) is the standard vectorized ReLU: elementwise, whichever of 0 or z is larger. **drelu** takes **z** (the PRE-activation, unlike Day 40's dtanh which took the POST-activation output) — ReLU's derivative is 1 wherever z was positive and 0 elsewhere, which **(z > 0)** computes directly as a boolean array, then **.astype(z.dtype)** converts True/False to 1.0/0.0.

```
def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)

def one_hot(y, num_classes):
    m = y.shape[0]
    encoded = np.zeros((m, num_classes))
    encoded[np.arange(m), y] = 1.0
    return encoded

def cross_entropy_loss(probs, y_onehot, eps=1e-9):
    probs = np.clip(probs, eps, 1 - eps)
    return -np.mean(np.sum(y_onehot * np.log(probs), axis=1))
```

Character-for-character identical to Days 39-40's **softmax**, **one_hot**, and **cross_entropy_loss** — today's lesson changes what happens BEFORE the classification head (adding conv+pool layers), not the classification head itself, so these three functions are simply reused unmodified.

```
def he_scale(n_in):
    return np.sqrt(2.0 / n_in)
```

Day 40's He/Kaiming initialization formula, reused here for BOTH the conv layer's filters and the dense hidden layer — a deliberate callback, since today's conv layer uses ReLU (Segment 1 above), the exact activation He initialization was designed for.

Segment 2 — conv2d_general and demo_convolution_basics (lines 90–142)

MATH

$$Z[i,j] = \text{sum over } (a,b) \text{ of } X[i \times stride + a,\ j \times stride + b] \times W[a,b] + b$$

```
def conv2d_general(X, W, b, stride=1, pad=0):
    if pad > 0:
        X = np.pad(X, pad, mode="constant")
    H, Win = X.shape
    kH, kW = W.shape
    out_H = (H - kH) // stride + 1
    out_W = (Win - kW) // stride + 1
    Z = np.zeros((out_H, out_W))
    for i in range(out_H):
        for j in range(out_W):
            patch = X[i*stride:i*stride+kH,
                      j*stride:j*stride+kW]
            Z[i, j] = np.sum(patch * W) + b
    return Z
```

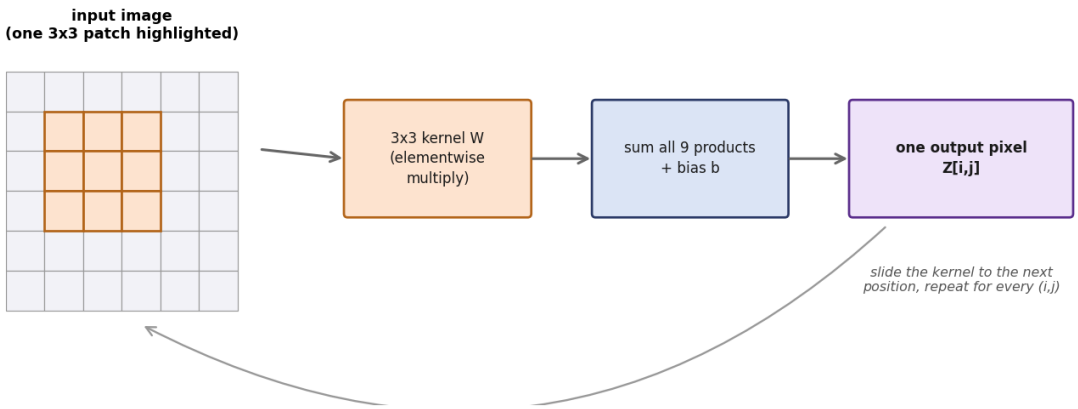
np.pad(X, pad, mode="constant") adds **pad** zeros around EVERY side of a 2D array by default — the simplest, most common padding choice. The nested **for i / for j** loop walks every valid output position; the slice **i*stride:i*stride+kH** is what makes STRIDE work — with stride=1 this steps one at a time, with stride=2 it jumps by 2, automatically skipping positions. This single-image, single-filter function exists only for this segment and Segment 3's diagnostics — the trainable ConvNet uses the batched, stride-1 version in Segment 4 instead.

```
img = np.zeros((size, size))
img[:, 3] = 1.0
img[:, 4] = 1.0
vertical_k = np.array([[-1, 0, 1],
                       [-1, 0, 1],
                       [-1, 0, 1]], dtype=float)
horizontal_k = vertical_k.T

Zv = conv2d_general(img, vertical_k, b=0.0)
Zh = conv2d_general(img, horizontal_k, b=0.0)
```

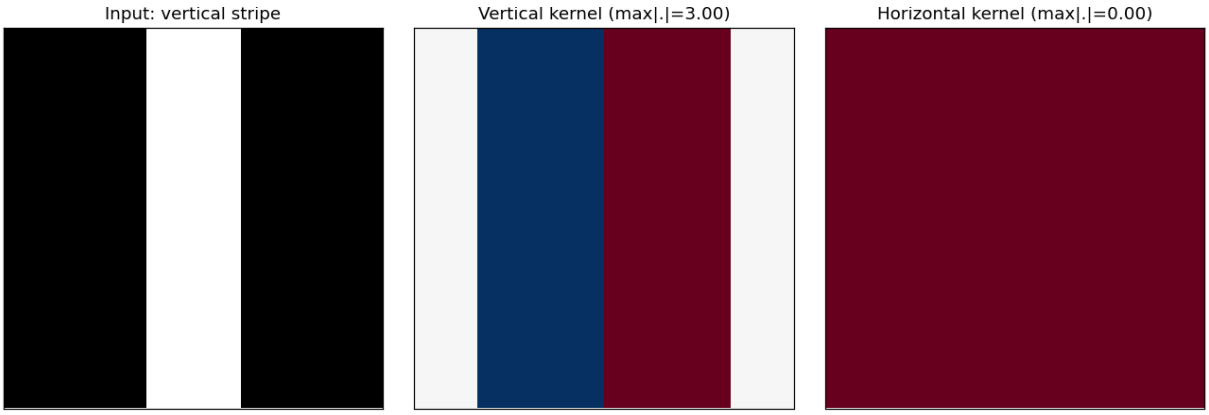
img[:, 3] = 1.0 and **img[:, 4] = 1.0** use column-slicing to paint a 2-pixel-wide bright vertical stripe using array assignment, not a loop. **vertical_k.T** is NumPy's transpose shorthand — swapping the kernel's rows and columns turns a vertical-edge detector into a horizontal-edge detector with zero new code, since the underlying pattern is identical, just rotated.

SYSTEM DIAGRAM



One convolution step: a 3x3 patch times the kernel, summed into one output number, then the kernel slides to the next position and repeats.

REAL GRAPH (script's own output)



Not a mockup — this is edge_detection.png, this exact script rerun fresh: the vertical kernel's strong response versus the horizontal kernel's flat, exactly-zero response.

Segment 3 — demo_padding_stride (lines 149–166)

MATH

$$out\_size = floor((H - k + 2 \times pad) / stride) + 1$$

```
def conv_output_size(H, k, pad=0, stride=1):
    return (H - k + 2 * pad) // stride + 1
```

A one-line function directly implementing the output-size formula — ***//*** is integer (floor) division, matching how a partial final kernel position simply doesn't count as a valid output position.

```
configs = [(0, 1), (1, 1), (0, 2)]
img = np.random.randn(H, H)
W = np.random.randn(k, k)
for pad, stride in configs:
    formula_size = conv_output_size(H, k, pad, stride)
    actual = conv2d_general(img, W, b=0.0,
                            stride=stride, pad=pad)
    print(f"...formula size={formula_size}, actual shape={actual.shape[0]}")
```

configs is a list of **(pad, stride)** TUPLES, unpacked directly in the **for pad, stride in configs** loop header — a compact way to test several parameter combinations without a nested loop. Each iteration computes the size TWO independent ways (the formula, and actually running the convolution) purely to demonstrate they agree.

Segment 4 — conv2d_forward, conv2d_backward, demo_feature_maps (lines 173–260)

MATH

Z[:,f,i,j] = sum(X[:,i:i+kH,j:j+kW] × W[f], per image) + b[f]

```
def conv2d_forward(X, W, b):
    m, H, Win = X.shape
    C_out, kH, kW = W.shape
    out_H = H - kH + 1
    out_W = Win - kW + 1
    Z = np.zeros((m, C_out, out_H, out_W))
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, i:i+kH, j:j+kW]
                Z[:, f, i, j] = np.sum(
                    patch * W[f], axis=(1, 2)) + b[f]
    return Z
```

Unlike Segment 2's single-image version, **X** here keeps its full BATCH dimension (**m**) throughout — **patch = X[:, i:i+kH, j:j+kW]** slices ALL m images' patches at once, and **axis=(1, 2)** sums over just the kernel's two spatial dimensions, leaving the batch dimension untouched. The outer **for f** loop adds one more level versus Segment 2: one full pass per FILTER, producing **C_out** feature maps instead of just one.

MATH

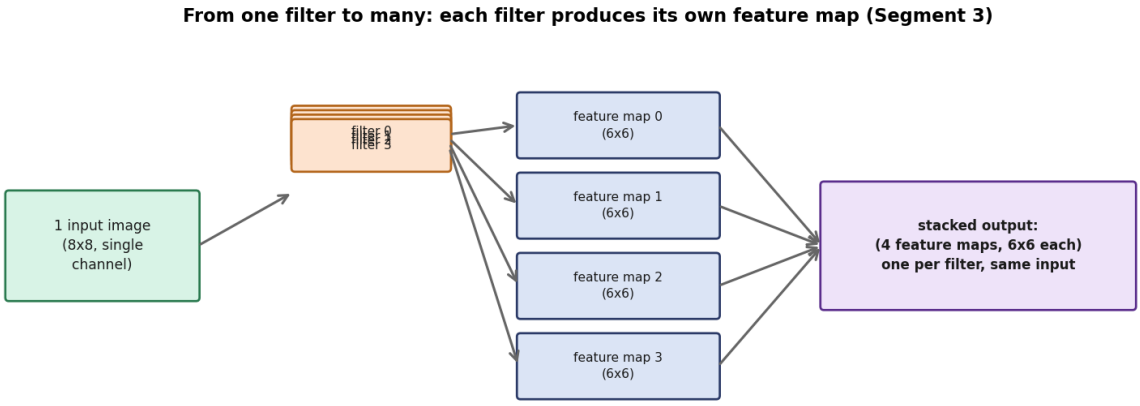
dW[f] = sum(dZ[:,f,i,j] × patch, per image) / m db[f] = sum(dZ[:,f,i,j]) / m
dX[patch region] += dZ[:,f,i,j] × W[f] (accumulated -- overlapping positions add up)

```
def conv2d_backward(dZ, X, W):
    ...
    dW = np.zeros_like(W)
    db = np.zeros(C_out)
    dX = np.zeros_like(X)
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, i:i+kH, j:j+kW]
                dZ_ij = dZ[:, f, i, j]
                dW[f] += np.sum(
                    dZ_ij[:,None,None]*patch, axis=0)/m
                db[f] += np.sum(dZ_ij) / m
                dX[:, i:i+kH, j:j+kW] += (
                    dZ_ij[:,None,None] * W[f])
```

dZ_ij[:,None,None] reshapes a (m,) array into (m,1,1) so it broadcasts correctly against the (m,kH,kW)-shaped **patch** — the same reshape-for-broadcasting trick used throughout this roadmap. Critically, **dX[..., i:i+kH, j:j+kW] += ...** uses **+=**, not **=** — because output positions OVERLAP (a single input pixel contributes to several

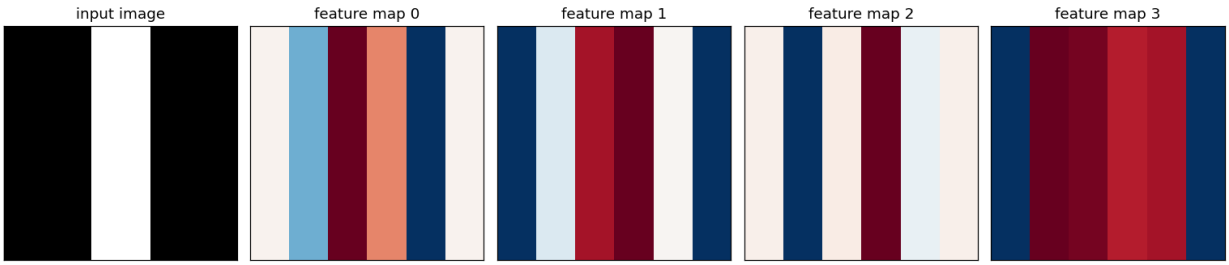
output positions when kH,kW > 1), each pixel's gradient must ACCUMULATE contributions from every position that used it, not just keep the last one written.

SYSTEM DIAGRAM



From one input image, 4 independent filters produce 4 independent feature maps, stacked into one layer's output.

REAL GRAPH (script's own output)



Not a mockup — this is feature_maps.png, this exact script rerun fresh: one image, 4 untrained filters, 4 distinct feature maps.

Segment 5 — maxpool_forward, maxpool_backward, demo_maxpool (lines 267–320)

```
    def maxpool_forward(A, size=2, stride=2):
        m, C, H, W = A.shape
        out_H = (H - size) // stride + 1
        out_W = (W - size) // stride + 1
        out = np.zeros((m, C, out_H, out_W))
        masks = {}
        for i in range(out_H):
            for j in range(out_W):
                window = A[:, :, i*stride:i*stride+size,
                               j*stride:j*stride+size]
                flat = window.reshape(m, C, size*size)
                idx = flat.argmax(axis=2)
                out[:, :, i, j] = flat[
                    np.arange(m)[:,None],
                    np.arange(C)[None,:], idx]
```

window.reshape(m, C, size*size) flattens each 2x2 (or larger) window into a 1D list of candidates per (image, channel) pair, so **.argmax(axis=2)** can find the WINNING position's flat index directly. **flat[np.arange(m)[:,None], np.arange(C)[None,:], idx]** is the same fancy-indexing pattern Day 39 used for one-hot encoding, extended to 3 dimensions: it pulls out exactly one value per (image, channel) pair — the one **idx** pointed to — in a single vectorized line.

MATH

mask[image, channel, argmax position] = 1, else 0 -- one 1 per window

```
                mask = np.zeros_like(flat)
                mm, cc = np.meshgrid(np.arange(m), np.arange(C),
                                     indexing="ij")
                mask[mm, cc, idx] = 1
                masks[(i, j)] = mask.reshape(m, C, size, size)
        return out, masks
```

np.meshgrid(..., indexing="ij") builds two grids **mm** and **cc** covering every (image, channel) COMBINATION, so **mask[mm, cc, idx] = 1** sets exactly one 1 per (image, channel) pair — at the argmax position found above — leaving every other entry 0. **masks** is a dict keyed by **(i, j)** window position, so the exact same mask used in the forward pass can be looked back up during **maxpool_backward** below.

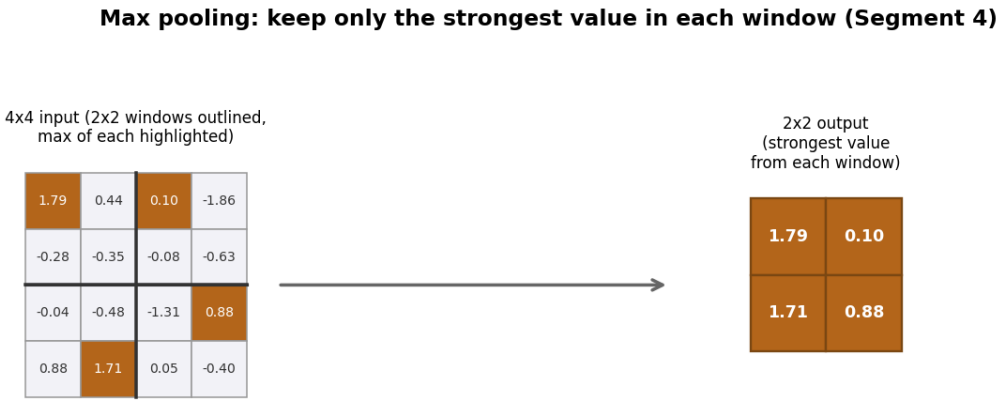
MATH

dA[window] += mask × dOut[i,j] (only the argmax position gets nonzero gradient)

```
def maxpool_backward(dOut, A, masks, size=2, stride=2):
    ...
    for i in range(out_H):
        for j in range(out_W):
            dA[:, :, i*stride:i*stride+size,
                    j*stride:j*stride+size] += (
                masks[(i, j)] * dOut[:, :, i, j][:, :, None, None]
            )
```

Multiplying the incoming gradient by **masks[(i, j)]** — the SAME mask built during the forward pass — routes the gradient to exactly the position that was the maximum, and zero everywhere else in the window, with no explicit if/else needed anywhere in this function.

SYSTEM DIAGRAM



Max pooling on a 4x4 grid with 2x2 windows: each window's single strongest value survives into the smaller 2x2 output.

Segment 6 — make_stripe_dataset and demo_dataset (lines 327–367)

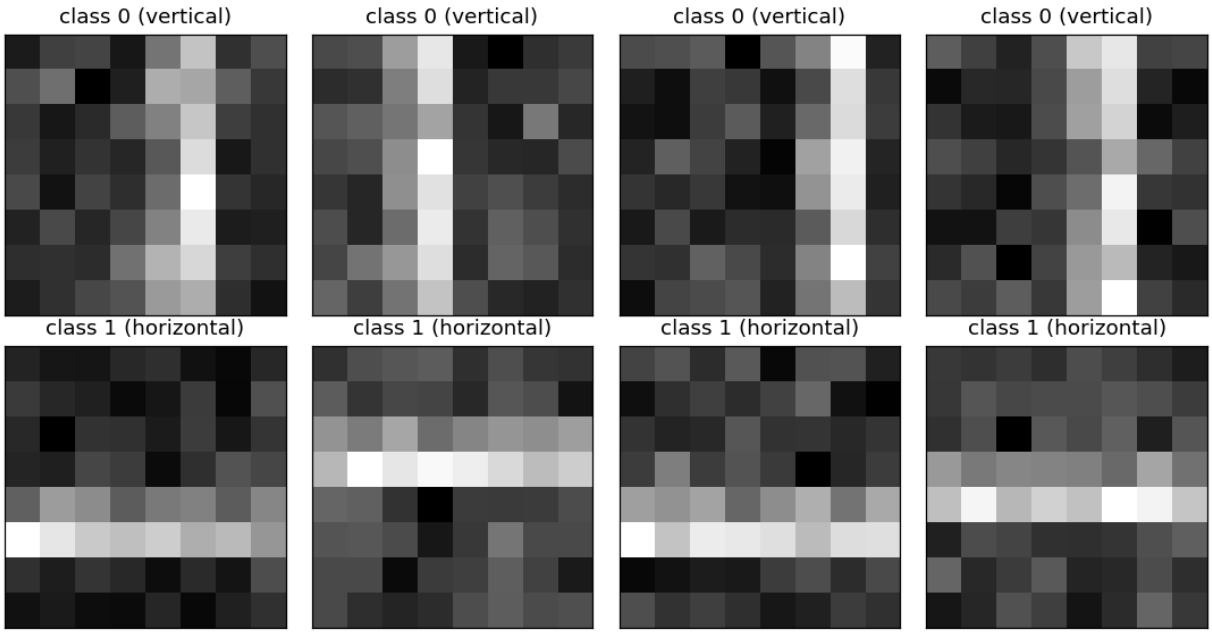
```
def make_stripe_dataset(n_per_class=150, size=8,
                        noise=0.3, seed=0):
    rng = np.random.RandomState(seed)
    X, y = [], []
    for _ in range(n_per_class):
        im = rng.randn(size, size) * noise
        col = rng.randint(1, size - 1)
        im[:, col] += 2.0
        im[:, col - 1] += 1.0
        X.append(im); y.append(0)
```

rng.randn(size, size) * noise starts every image as pure Gaussian background noise before any stripe is added. **rng.randint(1, size - 1)** picks a RANDOM column for the stripe (never the very first or last column, keeping the full stripe visible within the image) — a DIFFERENT random column every single image, so the network can't simply memorize 'the stripe is always at column 3.'

```
    for _ in range(n_per_class):
        im = rng.randn(size, size) * noise
        row = rng.randint(1, size - 1)
        im[row, :] += 2.0
        im[row - 1, :] += 1.0
        X.append(im); y.append(1)
    X = np.array(X); y = np.array(y)
    idx = rng.permutation(len(y))
    return X[idx], y[idx]
```

The class-1 loop is the exact mirror of class-0's, with row-slicing (**im[row, :]**) instead of column-slicing. **rng.permutation(len(y))** shuffles the FINAL combined dataset — without this, all 150 class-0 images would come before all 150 class-1 images, an ordering the training loop (Segment 10) doesn't shuffle again itself since it's full-batch, not mini-batch.

REAL GRAPH (script's own output)



Not a mockup — this is stripe_dataset.png, this exact script rerun fresh: 4 real example images from each class.

Segment 7 — ConvNet.__init__ (lines 375–392)

```
class ConvNet:
    def __init__(self, n_filters=4, k=3, img_size=8,
                 hidden=16, n_classes=2, lr=0.1, seed=42):
        rng = np.random.RandomState(seed)
        fan_in_conv = k * k
        self.Wc = rng.randn(n_filters, k, k) * (
            he_scale(fan_in_conv))
        self.bc = np.zeros(n_filters)
```

fan_in_conv = k * k treats a 3x3 kernel's fan-in as 9 — the number of input values ONE output pixel depends on, which is exactly the quantity He initialization's formula (Day 40) expects, even though this is a convolutional rather than dense layer. **self.Wc** is shaped **(n_filters, k, k)** — the same shape used throughout Segment 4's **conv2d_forward**.

```
        out_conv = img_size - k + 1
        pool_out = out_conv // 2
        flat_dim = n_filters * pool_out * pool_out
        self.W1 = rng.randn(flat_dim, hidden) * (
            he_scale(flat_dim))
        self.b1 = np.zeros(hidden)
        self.W2 = rng.randn(hidden, n_classes) * (
            he_scale(hidden))
        self.b2 = np.zeros(n_classes)
        self.lr = lr
```

out_conv, then **pool_out**, then **flat_dim** chain together the exact arithmetic needed to correctly size the first dense layer's INPUT dimension, computed once here rather than hardcoded — change **img_size**, **k**, or **n_filters** and **W1**'s shape adjusts automatically. **W1** and **W2** reuse Day 40's exact He-initialized dense-layer pattern.

Segment 8 — ConvNet.forward (lines 394–404)

```
            def forward(self, X):
                m = X.shape[0]
                self.X = X
                self.Zc = conv2d_forward(X, self.Wc, self.bc)
                self.Ac = relu(self.Zc)
                self.pooled, self.masks = maxpool_forward(
                    self.Ac, 2, 2)
                self.flat = self.pooled.reshape(m, -1)
```

Five lines, five layers: **self.Zc** (conv), **self.Ac** (ReLU), **self.pooled** (max pool, plus **self.masks** saved for the backward pass), and **self.flat**. **self.pooled.reshape(m, -1)** is the FLATTEN step — **-1** tells NumPy to infer the remaining dimension automatically (here, **n_filters * pool_out * pool_out**), collapsing the (m, C, H, W)-shaped pooled output into a plain (m, flat_dim) matrix the dense layers below can use.

```
                self.z1 = self.flat @ self.W1 + self.b1
                self.a1 = tanh(self.z1)
                self.z2 = self.a1 @ self.W2 + self.b2
                self.probs = softmax(self.z2)
                return self.probs
```

Character-for-character identical to Day 40's DeepNet dense-layer forward code — the exact same **tanh** hidden activation and **softmax** output, just fed **self.flat** (the conv+pool layers' output) instead of raw input.

Segment 9 — ConvNet.backward (lines 406–424)

```
            def backward(self, y_onehot):
                m = y_onehot.shape[0]
                dz2 = self.probs - y_onehot
                dW2 = self.a1.T @ dz2 / m
                db2 = dz2.mean(axis=0)
                da1 = dz2 @ self.W2.T
                dz1 = da1 * dtanh(self.a1)
                dW1 = self.flat.T @ dz1 / m
                db1 = dz1.mean(axis=0)
                dflat = dz1 @ self.W1.T
```

The first nine lines are, again, character-for-character identical to Day 40's DeepNet backward pass — **dz2 = probs - y_onehot**, then the standard dense-layer gradient chain down to **dflat**, the gradient with respect to the FLATTENED conv+pool output.

MATH

$$dpooled = reshape(dflat, pooled.shape)$$
$$dAc = maxpool\_backward(dpooled, ...) \quad dZc = dAc \times ReLU'(Zc)$$

```
                dpooled = dflat.reshape(self.pooled.shape)
                dAc = maxpool_backward(
                    dpooled, self.Ac, self.masks, 2, 2)
                dZc = dAc * drelu(self.Zc)
                dWc, dbc, _ = conv2d_backward(
                    dZc, self.X, self.Wc)
```

dflat.reshape(self.pooled.shape) is the FLATTEN step's backward pass — reshaping a gradient is its own inverse, so this simply restores the (m, C, H, W) shape **maxpool_backward** expects. Note the discarded _ in **dWc, dbc, _ = conv2d_backward(...)** — the third return value (**dX**, gradient w.r.t. the input IMAGE) is never used, because this is the network's first layer; there's no earlier layer to pass that gradient back into.

```
                self.W2 -= self.lr * dW2
                self.b2 -= self.lr * db2
                self.W1 -= self.lr * dW1
                self.b1 -= self.lr * db1
                self.Wc -= self.lr * dWc
                self.bc -= self.lr * dbc
```

One plain gradient-descent update line per learnable parameter — six in total, covering both dense layers AND the conv layer's filters, all using the identical **param -= lr * gradient** rule seen throughout every prior day.

Segment 10 — train, accuracy, demo_training (lines 426–479)

```
        def train(self, X, y, epochs, num_classes=2):
            y_onehot = one_hot(y, num_classes)
            losses = []
            for _ in range(epochs):
                probs = self.forward(X)
                losses.append(
                    cross_entropy_loss(probs, y_onehot))
                self.backward(y_onehot)
            return losses

        def accuracy(self, X, y):
            probs = self.forward(X)
            return (probs.argmax(axis=1) == y).mean()
```

Deliberately simple, full-batch-only training loop — no mini-batching option here, unlike Day 39's SoftmaxNet — since today's lesson isolates convolution and pooling as the variables under test. **accuracy** is identical to every prior day's: **argmax** picks the most-confident class, compared against the true labels.

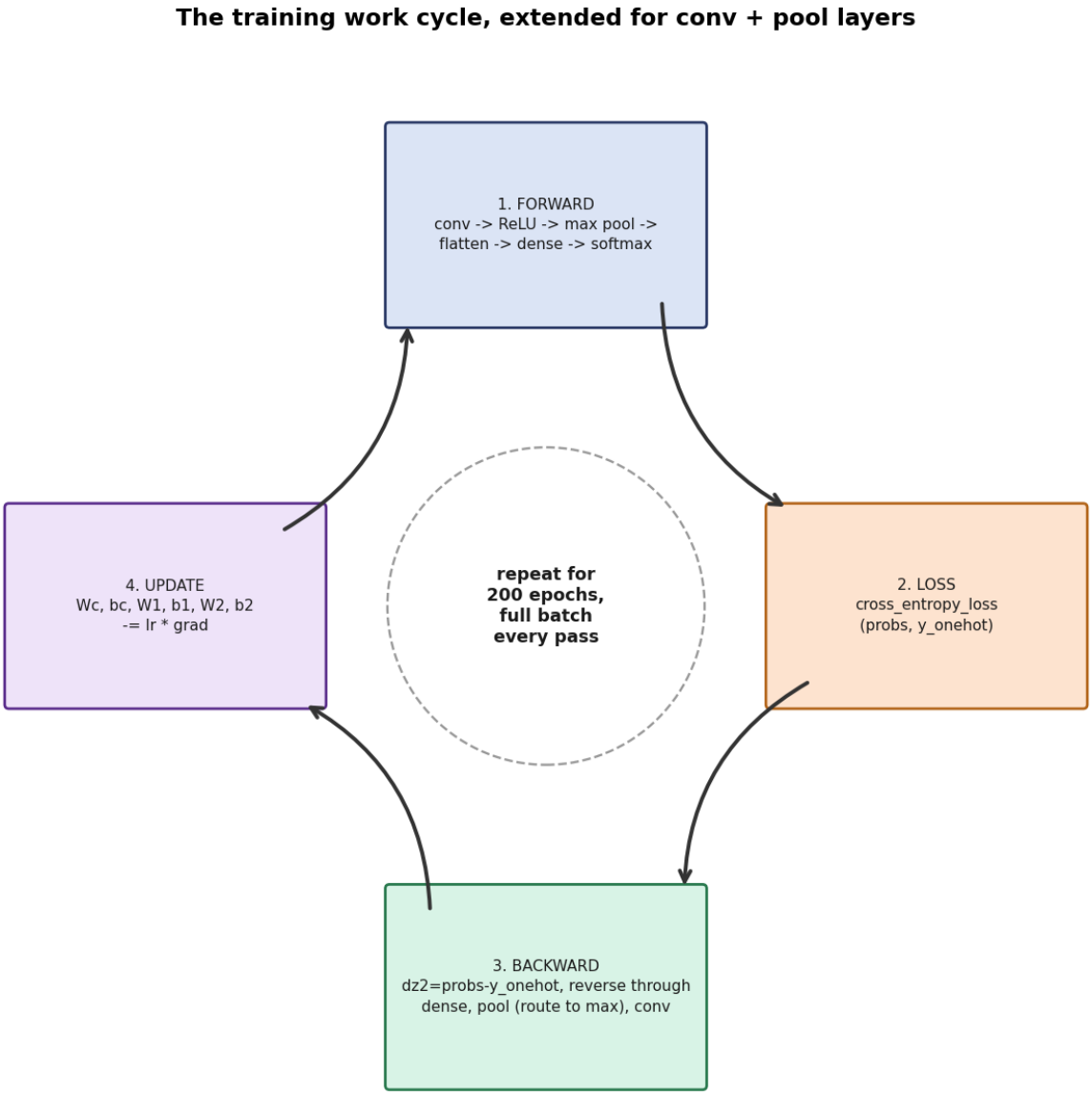
```
    net = ConvNet(n_filters=4, k=3, img_size=8, hidden=16,
                  n_classes=2, lr=0.1, seed=42)
    untrained_acc = net.accuracy(X, y)

    losses = net.train(X, y, epochs=200)
    train_acc = net.accuracy(X, y)

    X_test, y_test = make_stripe_dataset(
        50, 8, 0.3, seed=999)
    test_acc = net.accuracy(X_test, y_test)
```

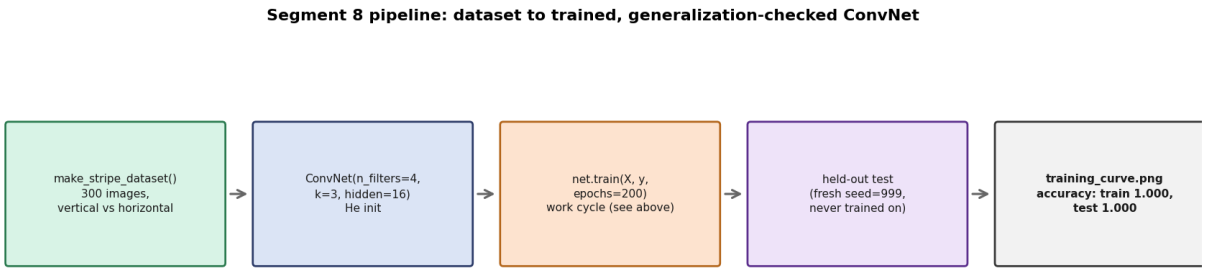
untrained_acc is measured BEFORE **net.train(...)** is ever called — capturing the network's genuinely random starting point. **make_stripe_dataset(50, 8, 0.3, seed=999)** generates a completely SEPARATE dataset with a different seed than the training data's **seed=42** — **test_acc** is measured on images that provably never appeared anywhere in the **net.train(...)** call above.

SYSTEM DIAGRAM



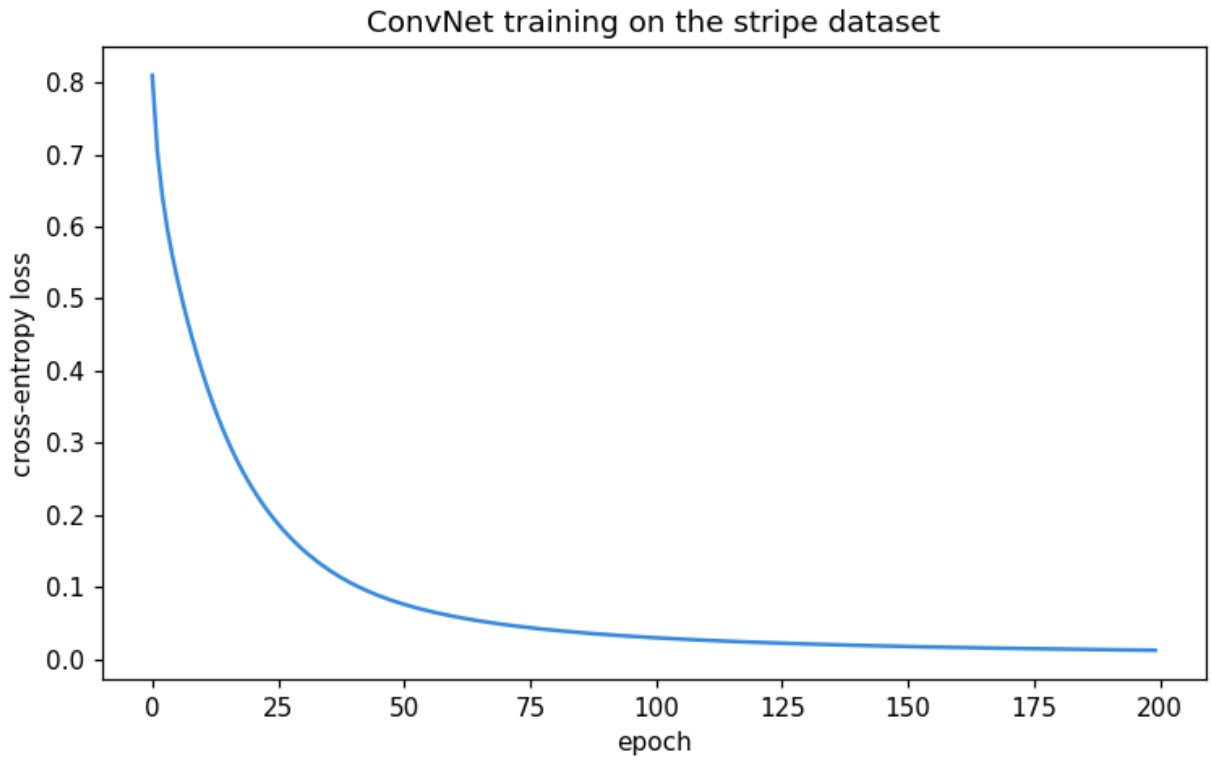
The training work cycle, extended for conv + pool layers: forward through conv/ReLU/pool/dense/softmax, loss, backward through the same chain in reverse, then update every learnable parameter.

SYSTEM DIAGRAM



Zooming out: the full pipeline this segment runs end to end, from the stripe dataset through training to a genuine held-out generalization check.

REAL GRAPH (script's own output)



Not a mockup — this is training_curve.png, this exact script rerun fresh: cross-entropy loss smoothly converging over 200 epochs.

Segment 11 — note_on_pytorch, main, and the Entry-Point Guard (lines 482–510)

```
        nn.Conv2d(in_channels=1, out_channels=4, kernel_size=3)
        nn.MaxPool2d(kernel_size=2, stride=2)
        nn.ReLU()
```

note_on_pytorch() is just **print()** calls containing plain text — not executable code, and PyTorch is never imported anywhere in this file, exactly like every prior day's equivalent function. These are reference lines mapping today's hand-written conv and pooling mechanisms to their PyTorch layer equivalents.

```
        def main():
            demo_convolution_basics()
            demo_padding_stride()
            demo_feature_maps()
            demo_maxpool()
            X, y = demo_dataset()
            demo_training(X, y)
            note_on_pytorch()

        if __name__ == "__main__":
            main()
```

The same overall structure as every prior day: one **main()** calling every demo in the exact order they should run, with **X, y** from **demo_dataset()** explicitly threaded into **demo_training(X, y)** — the same shared-data pattern Day 39 used to guarantee a fair comparison — guarded by the standard **if __name__ == "__main__":** check.

Quick Syntax Glossary — New or Reinforced Constructs

`np.pad(X, pad, mode="constant")` — Adds a border of zeros (by default) around every side of an array -- the standard way to implement padding before a convolution.

`i*stride:i*stride+kH slicing` — The slice expression that makes stride work: with stride=1 it steps one position at a time; with stride=2 it jumps by 2, automatically skipping every other position.

`array.T (transpose)` — Swaps an array's rows and columns -- used here to turn a vertical-edge kernel into a horizontal-edge kernel with zero new code, since the pattern is the same, just rotated.

`configs = [(0,1), (1,1), (0,2)] unpacking` — A list of tuples, unpacked directly in a for pad, stride in configs: loop header -- tests several parameter combinations without a nested loop.

`dZ_ij[:, None, None] broadcasting` — Reshapes a (m,) array into (m,1,1) so it broadcasts correctly against a (m,kH,kW)-shaped patch -- the same reshape-for-broadcasting trick used throughout this roadmap.

`dX[...] += ... (accumulation, not =)` — Because output positions overlap in a convolution, each input pixel's gradient must accumulate contributions from every output position that used it -- using = instead of += would silently lose contributions.

`window.reshape(m, C, size*size) + argmax(axis=2)` — Flattens each pooling window into a 1D list of candidates per (image, channel) pair so argmax can find the winning position's flat index directly.

`np.meshgrid(..., indexing="ij")` — Builds coordinate grids covering every (image, channel) combination -- used to set exactly one 1 per pair in a mask array, generalizing Day 39's one-hot fancy-indexing to more dimensions.

`array.reshape(m, -1) (flatten)` — -1 tells NumPy to infer the remaining dimension automatically -- collapses a (m, C, H, W) array into a plain (m, flat_dim) matrix a dense layer can use.

`discarded return value (dWc, dbc, _ = ...)` — The underscore _ is a conventional placeholder for a return value that's computed but never used -- here, the gradient w.r.t. the input image, irrelevant for a first layer with nothing earlier to pass it to.

End of Day 41 syntax reference. For the convolution and pooling concepts these lines implement, see the main Day 41 study guide PDF.